

ESE 224 – Final Project

StockSim – Historical Market Analyzer & Trading Strategy Simulator

Course: ESE 224 – Advanced Data Structures and Algorithms

Deadline: 3 weeks from project release

Team Size: Individual

Total Points: 100 + 10 Bonus

AI Feedback Required: After completing this project, every student **must** fill out the ESE 224 AI Usage Feedback Form:

https://docs.google.com/forms/d/1NpkLKFrnvGgSsysC4Ba0lhCPSL_MIHtGaU-X84D5CnY This form is **mandatory** and counts toward your AI Prompts

Documentation grade. It takes ~5 minutes and asks how you used AI, whether it helped or misled you, and what you learned from the process.

Overview

In this project you will build **StockSim**, a C++ command-line application that:

1. Loads 25 years of historical stock data from Yahoo Finance CSV files
2. Organizes that data using the custom data structures you built throughout this course
3. Simulates four distinct investment strategies and backtests each against real market history
4. Answers the open-ended research question: **“Does a Dynamic SIP (invest more on dips) outperform a Fixed SIP over 25 years of S&P 500 data?”**

This project intentionally connects every module of the course to a practical, real-world domain.

You will use **linked lists, queues, stacks, binary search trees, vectors, iterators, templates, and OOP** — not as isolated exercises, but as integrated components of one working system.

Restrictions

- **No STL containers for custom data structures.** You may NOT use `std::queue`, `std::stack`, `std::list`, `std::deque`, or `std::map`/`std::unordered_map`. You **MUST** implement these by hand using raw pointers or arrays.

- **You MAY use:** `std::vector` (for Portfolio holdings only), `std::string`, `std::fstream`, `std::iostream`, `std::stringstream`, `std::algorithm` (sort/find only), and `std::io manip`.
- **No hash tables.** No `std::unordered_map`, `std::unordered_set`, or any hash-based structure.
- **No graphs.** No graph data structure or graph algorithm.
- **No external libraries.** No Boost, no TA-Lib, no pandas equivalent. Everything is built from scratch.
- **Header guards required** on every `.h` file: `#ifndef`, `#define`, `#endif`.
- **All dynamic memory must be explicitly managed.** No smart pointers (`unique_ptr`, `shared_ptr`). Use `new` and `delete` correctly. Destructors must free all allocated memory.

Data

The CSV files are provided in your starter kit `data/` folder. **Do not download different files** — all teams must use exactly these datasets to ensure fair comparison on the leaderboard.

Ticker	Description	File	Column Format
SPX	S&P 500 Index (primary dataset)	<code>data/SPX.csv</code>	Date, Open, High, Low, Close, Adj Close, Volume
AMZN	Amazon Inc.	<code>data/AMZN.csv</code>	Date, Open, High, Low, Close, Adj Close, Volume
NVDA	NVIDIA Corporation (high volatility)	<code>data/NVidia_stock_history.csv</code>	Date, Adj Close, Close, High, Low, Open, Volume

Date range: restrict all analysis to 2000-01-01 through 2020-01-01 (20 years). Your `CSVParser` must filter out rows outside this range. Do not use data before 2000 or after 2020 — results outside this window will not be counted toward the leaderboard.

Important — NVDA column order is different. `NVidia_stock_history.csv` has columns in the order Date, Adj Close, Close, High, Low, Open, Volume (not the standard Yahoo order). Your `CSVParser::loadHistory()` must detect and handle both column layouts. One approach: read the header row and map column names to indices dynamically.

Important: All datasets skip weekends and market holidays. Your code must handle non-consecutive dates — do not assume one linked-list node = one calendar day.

Class Architecture

You must implement the following classes. Each class maps to one or more course modules.

1. FinancialAsset — Abstract Base Class (OOP / ADT)

File: include/FinancialAsset.h, src/FinancialAsset.cpp

Abstract base class for all financial instruments. Must contain: - Protected member variables: ticker (string), name (string), sector (string) - Pure virtual methods: - virtual void printSummary() const = 0 - virtual double calculateAnnualReturn(int year) const = 0 - virtual string getType() const = 0 - Non-virtual: getTicker(), getName(), setSector(), getSector() - Virtual destructor

2. Stock — Derived Class (OOP / Linked List / Iterators)

File: include/Stock.h, src/Stock.cpp

Represents a single stock (e.g., AMZN). Inherits from FinancialAsset. Must contain: - A PriceHistory* member — owns the price history linked list - loadFromCSV(const string& filename) — parses CSV and builds the linked list - getHistory() — returns pointer to the PriceHistory - getPriceOnDate(const string& date) const — O(n) scan - calculateAnnualReturn(int year) const — override: (end_price - start_price) / start_price * 100 - printSummary() const — override: print ticker, date range, total return, number of trading days - getType() const — returns "Stock"

3. ETF — Derived Class (OOP)

File: include/ETF.h, src/ETF.cpp

Represents an index instrument (e.g., SPX). Inherits from FinancialAsset. Must contain: - Everything in Stock, plus: - expenseRatio (double) member - getType() const — returns "ETF" - printSummary() const — override: additionally prints expense ratio and 10-year CAGR

4. PriceNode — Linked List Node

File: include/PriceNode.h (header only, no .cpp needed)

Node for the doubly-linked price history list:

```
struct PriceNode {
    string date;           // "YYYY-MM-DD"
    double open;
    double high;
    double low;
    double close;
    long volume;
    PriceNode* next;
```

```
    PriceNode* prev;
};
```

5. PriceHistory — Doubly-Linked List with Iterators (Linked List / Iterators)

File: include/PriceHistory.h, src/PriceHistory.cpp

A doubly-linked list of PriceNode items. Must contain: - PriceNode* head, PriceNode* tail, int size - void append(const string& date, double open, double high, double low, double close, long volume) - PriceNode* findByDate(const string& date) const — linear search - void printRange(const string& startDate, const string& endDate) const - int getSize() const - Destructor: frees all nodes

Forward Iterator (inner class):

```
class Iterator {
public:
    Iterator(PriceNode* node);
    Iterator& operator++();    // pre-increment
    PriceNode& operator*();
    bool operator!=(const Iterator& other) const;
};
Iterator begin() const;
Iterator end() const;
```

Reverse Iterator (inner class):

```
class ReverseIterator {
public:
    ReverseIterator(PriceNode* node);
    ReverseIterator& operator++();    // moves backward through list
    PriceNode& operator*();
    bool operator!=(const ReverseIterator& other) const;
};
ReverseIterator rbegin() const;
ReverseIterator rend() const;
```

The reverse iterator must walk from tail toward head using prev pointers.

6. CircularQueue — Fixed-Size Circular Queue (Queues / Moving Averages)

File: include/CircularQueue.h, src/CircularQueue.cpp

A fixed-size circular buffer for computing N-day moving averages. Must contain: - double* buffer, int capacity, int head, int tail, int count - void enqueue(double value) — if full, overwrites oldest (circular behavior) - double dequeue() - double peek() const — returns oldest value without removing - double getAverage() const — $O(n)$ sum / count - bool isFull() const, bool isEmpty() const - int getCount() const - Constructor takes capacity: CircularQueue(int capacity) - Destructor frees buffer

Usage: To compute a 50-day moving average, create `CircularQueue(50)`. Enqueue each day's close price. Once full, `getAverage()` returns the 50-day MA.

7. TradeStack — LIFO Stack for Undo (Stacks)

File: `include/TradeStack.h`, `src/TradeStack.cpp`

Pointer-based stack for trade history (supports undo). Must contain:

```
struct TradeRecord {
    string ticker;
    string date;
    double price;
    int    shares;
    string action; // "BUY" or "SELL"
    double totalCost;
};
```

- Node-based stack: `struct StackNode { TradeRecord data; StackNode* next; };`
- `void push(const TradeRecord& record)`
- `TradeRecord pop()`
- `TradeRecord peek() const`
- `bool isEmpty() const`
- `int getSize() const`
- `void printAll() const` — print entire trade history top to bottom
- Destructor frees all nodes

8. OrderQueue — FIFO Queue for Pending Orders (Queues)

File: `include/OrderQueue.h`, `src/OrderQueue.cpp`

Node-based FIFO queue for pending trade orders. Must contain:

```
struct Order {
    string ticker;
    string type; // "MARKET", "LIMIT"
    string side; // "BUY", "SELL"
    double targetPrice; // used for LIMIT orders; 0 for MARKET
    int    shares;
    string submittedDate;
};
```

- `struct QueueNode { Order data; QueueNode* next; };`
- `void enqueue(const Order& order)`
- `Order dequeue()`
- `Order peek() const`
- `bool isEmpty() const`

- int getSize() const
- void printAll() const
- Destructor frees all nodes

9. StockBST — Binary Search Tree by Return (Trees)

File: include/StockBST.h, src/StockBST.cpp

A BST that indexes stocks by a double key (annual return % or volatility). Must contain:

```
struct BSTNode {
    string ticker;
    double key;           // return % or volatility
    int    year;          // which year this stat is for
    BSTNode* left;
    BSTNode* right;
};
```

- void insert(const string& ticker, double key, int year)
- BSTNode* search(double key) const — exact match
- void rangeSearch(double low, double high, vector<BSTNode*>& results) const — collect all nodes where low <= key <= high
- void inorder() const — print sorted by key
- void preorder() const
- void postorder() const
- int getHeight() const — recursive
- void clear() — frees all nodes
- Destructor calls clear()

10. Portfolio — Holdings Manager (Vectors / OOP Integration)

File: include/Portfolio.h, src/Portfolio.cpp

Manages a user's stock portfolio. Must contain:

```
struct Position {
    string ticker;
    int    shares;
    double avgCostBasis; // average purchase price
    double currentPrice; // updated when refreshed
};
```

- vector<Position> holdings (std::vector is allowed here)
- OrderQueue pendingOrders
- TradeStack tradeHistory
- double cashBalance

Methods: -void buyShares(const string& ticker, int shares, double price, const string& date) -void sellShares(const string& ticker, int shares, double price, const string& date) -void undoLastTrade() — pops TradeStack and reverses the transaction -void queueOrder(const Order& order) — add to OrderQueue -void executeNextOrder(double currentPrice) — dequeue and execute if conditions met -double getTotalValue() const -double getTotalReturn() const -void printHoldings() const -void sortHoldingsByReturn() — sort vector by unrealized gain/loss (use std::sort with comparator) -void sortHoldingsByTicker() — alphabetical

11. TradingStrategy — Abstract Strategy Base (OOP / Polymorphism)

File: include/TradingStrategy.h, src/TradingStrategy.cpp

```
struct SimResult {
    string strategyName;
    double finalValue;
    double totalInvested;
    double totalReturn;      // percentage
    double cagr;             // Compound Annual Growth Rate
    double maxDrawdown;     // worst peak-to-trough drop
    int    totalTrades;
};
```

- Pure virtual: virtual SimResult backtest(PriceHistory* history, double monthlyCapital, int startYear, int endYear) = 0
- Pure virtual: virtual string getName() const = 0
- Non-virtual: void printResult(const SimResult& result) const
- Non-virtual helper: double calculateCAGR(double startVal, double endVal, int years) const
- Non-virtual helper: double calculateMaxDrawdown(const vector<double>& portfolioValues) const
- Virtual destructor

12. FixedSIPStrategy — Derived Strategy (OOP)

File: include/FixedSIPStrategy.h, src/FixedSIPStrategy.cpp

Invests a fixed dollar amount on the first trading day of each month. - backtest(...) — iterate through PriceHistory month by month, buy fractional shares at close price - getName() — returns “Fixed SIP”

13. DynamicSIPStrategy — Derived Strategy (OOP / Open-Ended)

File: include/DynamicSIPStrategy.h, src/DynamicSIPStrategy.cpp

Your custom investment algorithm — the only strategy in this project where **you define the logic**.

The baseline you are trying to beat: FixedSIPStrategy invests the same dollar amount on the same day every month, regardless of market conditions.

Your freedom: you may design any algorithm that uses the daily price data available in PriceHistory. Some dimensions to consider (not a checklist — think beyond these): - *When* within the month to invest — not necessarily the 1st day - *How much* to invest each month — vary based on market signals - *Whether* to skip a month and carry funds forward - Any signal you can derive from price, volume, moving averages, or trend direction

Hard constraints: - Total capital deployed over the full period must equal $(\text{months}) \times \text{monthlyCapital}$ — budget-neutral relative to Fixed SIP - You may only use data available up to the current trading day (no lookahead in the linked list) - No external libraries — only the data structures you built in this project

You define your own constructor and member variables. The only interface requirement is implementing `backtest()` and `getName()` (inherited from `TradingStrategy`).

- `backtest(history, monthlyCapital, startYear, endYear)` — your full algorithm
- `getName()` — include your key parameters in the string so results are identifiable

14. GoldenCrossStrategy — Derived Strategy (OOP / Circular Queue)

File: `include/GoldenCrossStrategy.h`, `src/GoldenCrossStrategy.cpp`

Classic technical analysis strategy: - Uses two `CircularQueue` objects: one for 50-day MA, one for 200-day MA - **Buy signal:** 50-day MA crosses above 200-day MA (Golden Cross) - **Sell signal:** 50-day MA crosses below 200-day MA (Death Cross) - Hold cash when no position; fully invested when signal triggers

- `backtest(...)` — iterate through all trading days; update both CQs daily; detect crossovers
- `getName()` — returns “Golden Cross (50/200 MA)”

15. MomentumStrategy — Derived Strategy (OOP)

File: `include/MomentumStrategy.h`, `src/MomentumStrategy.cpp`

Buys when recent momentum is positive, sells/stays cash when negative: - Every month, compute 6-month trailing return using `ReverseIterator` on `PriceHistory` - Buy if trailing return > `momentumThreshold`; sell if below 0 - Rebalance monthly

- Constructor: `MomentumStrategy(double momentumThreshold)`
- `backtest(...)` — override
- `getName()` — returns “6-Month Momentum”

16. StockManager<T> — Template Class (Templates / Generic Programming)

File: `include/StockManager.h` (header only – template must be fully defined in header)

A generic manager for any collection of FinancialAsset-derived objects:

```
template <typename T>
class StockManager {
    vector<T*> assets;
public:
    void addAsset(T* asset);
    void removeAsset(const string& ticker);
    T* findByTicker(const string& ticker) const;
    void sortByAnnualReturn(int year);    // sort assets vector by
    calculateAnnualReturn(year)
    void sortByTicker();
    void printAll() const;
    int getCount() const;
    ~StockManager();    // must delete all T* pointers
};
```

Usage in main: StockManager<Stock> for stocks, StockManager<ETF> for ETFs.

17. CSVParser — Utility Class

File: include/CSVParser.h, src/CSVParser.cpp

Static utility class for loading Yahoo Finance CSVs: - static PriceHistory* loadHistory(const string& filename) — opens file, skips header row, parses each line into a PriceNode, returns populated PriceHistory* - static bool dateInRange(const string& date, const string& start, const string& end) - static int extractYear(const string& date) — parse “YYYY-MM-DD” → int year - static int extractMonth(const string& date) — parse → int month

The CSV parser must handle: - Missing/malformed rows (skip them) - Weekend/holiday gaps (store whatever rows exist; don't fill in missing dates) - Filter rows outside the required date window (2000-01-01 to 2020-01-01)

Main Program — Menu System

File: main.cpp

The main function must: 1. Print a login prompt: ask for student name and ID (no actual validation needed, just store it) 2. Load CSV data on startup (or from menu) 3. Provide the following menu:

```
===== StockSim: Historical Market Analyzer =====
Student: [name] | ID: [id]

[1] Load stock data from CSV
[2] Display price history (linked list traversal)
[3] Search by date range
[4] Find stocks by return range (BST range query)
```

```

[5]  Insert stock into performance BST
[6]  Display BST (inorder / preorder / postorder)
[7]  Add stock to portfolio
[8]  Remove stock from portfolio
[9]  Queue a pending order
[10] Execute next pending order
[11] Undo last trade
[12] Run strategy simulation (choose strategy)
[13] Compare all strategies head-to-head
[14] Display portfolio summary
[15] Display full trade history
[0]  Exit

```

Enter choice:

Menu item 12 must present a sub-menu to choose strategy and configure parameters (monthly capital, start year, end year, and strategy-specific parameters).

Menu item 13 runs all four strategies with the same parameters and prints a comparison table.

Grading Rubric

Component	Points	Key Requirements
FinancialAsset / Stock / ETF (OOP)	10	Abstract base, virtual methods, polymorphic printSummary, correct inheritance
PriceHistory + Iterators (Linked List)	15	Doubly-linked with next/prev, forward iterator, reverse iterator using prev pointers
CircularQueue + OrderQueue + TradeStack (Queue/Stack)	15	All hand-rolled with pointers/arrays; no STL; correct FIFO/LIFO/circular behavior
StockBST (Trees)	15	Insert, search, rangeSearch, all 3 traversals, getHeight
TradingStrategy hierarchy (OOP + Open-Ended)	15	All 4 strategies work correctly; Dynamic SIP correctly adjusts investment; CAGR and maxDrawdown computed
StockManager (Templates)	10	Fully generic, works with both Stock and ETF, sortByAnnualReturn uses polymorphism
Main + Menu	10	All 15 menu items functional; clean UX;

		student login
Report (PDF, max 20 pages)	20	See Report Requirements below
AI Prompts per Section	10	See AI Documentation Requirements below
BONUS: DynamicSIP Leaderboard	Top 3 only: +10 / +5 / +3	Best DynamicSIP final value on SPX 2000–2020 across class

Total: 100 points + 10 bonus

Report Requirements (20 points, PDF, max 20 pages)

The report must contain:

1. **Introduction** (1–2 pages): Problem statement, motivation, dataset description
2. **Data Structure Design** (4–5 pages): For each data structure (linked list, BST, circular queue, stack, queue), explain:
 - Your design decisions
 - Time complexity of key operations (with justification)
 - Why this structure is the right choice for this application
3. **Trading Strategies** (3–4 pages): Describe each strategy’s logic, parameters, and expected behavior
4. **Results and Analysis** (4–5 pages):
 - Backtesting results for all 4 strategies on SPX (2000–2020)
 - Comparison table: final value, CAGR, max drawdown, total trades
 - Does Dynamic SIP beat Fixed SIP? Under what conditions?
 - Include results for at least one volatile stock (NVDA or AMZN) vs SPX
5. **Conclusion** (1 page): Lessons learned, limitations, what you would improve

AI Documentation Requirements (10 points)

In **each section of your report**, include a sidebar or footnote box titled “**AI Usage Log**” that lists:

- The exact prompt(s) you used in that section
- What the AI returned (one-sentence summary)
- What you changed or corrected, and why

Example format for each entry:

[AI Usage – Section 2: PriceHistory]

Prompt: "Explain how to implement a reverse iterator on a doubly linked list in C++"

AI Output: Explained using a wrapper class with prev pointer traversal

Modification: AI used `std::iterator` base class which is not allowed in this course.

Rewrote without inheriting from `std::iterator`.

If you used AI for a section, document it. If you did not use AI for a section, write “No AI used.” Both are acceptable — the points come from honesty and reflection, not from avoiding or maximizing AI use.

Mandatory Form: After completing this project, fill out the ESE 224 AI Usage Feedback Form:

https://docs.google.com/forms/d/1NpkLKFrnvGgSsyncC4Ba0lhCPSL_MIHtGaU-X84D5CnY This form asks: how often you used AI, at which stages, prompt

quality, whether AI misled you, and your overall learning impact. **Completion of this form is required to receive full credit on the AI Prompts section.**

Bonus: Dynamic SIP Algorithm Leaderboard

This bonus is **competitive** — only the top 3 teams across the entire class earn points.

The Challenge

Design the best-performing DynamicSIPStrategy you can over the 20-year window (2000-01-01 to 2020-01-01) on **SPX data**.

The only rules: - Total capital deployed over the full 20 years must equal $240 \times \text{monthlyCapital}$ (same total as Fixed SIP — budget-neutral) - No lookahead: your algorithm can only use price data up to and including the current day - No external libraries

Everything else is open. Change when you invest, how much, skip months, use moving averages, react to volume — it is entirely up to you.

Scoring

Rank	Bonus Points
1st place (highest final portfolio value)	+10 points
2nd place	+5 points
3rd place	+3 points
All other teams	No bonus

Submission

In addition to your standard submission, include a file `bonus_strategy.txt` in your project root containing:

```
Strategy Name      : [your getName() output]
monthlyCapital     = $500
startYear          = 2000
endYear            = 2020
ticker             = SPX

Algorithm Description :
[2-4 sentences explaining the logic of your strategy in plain English]

Parameters Used      :
[list each parameter name and value your constructor takes]

Final Portfolio Value : $XX,XXX.XX
Fixed SIP Baseline    : $XX,XXX.XX   (for comparison)
CAGR                  : X.XX %
Max Drawdown          : X.XX %
Total Trades          : XXX
```

Also include `bonus_sweep.png` in your output/ folder showing your program's simulation output.

Leaderboard

Results will be collected and **posted publicly for the entire class** after the submission deadline. Every team's final portfolio value will be ranked. The top three will be announced and receive bonus points on their final grade.

Same data, same starting capital, same 20-year window for everyone. The only difference is the algorithm each team writes. Think about what the 2000–2020 period looked like: a dot-com crash (2000–2002), a recovery, a financial crisis (2008), and then a decade-long bull run (2010–2020). A strategy that does well across all three phases will rank higher than one that only exploits one of them.

3-Week Timeline (Suggested)

Week	Focus
Week 1	CSV parser, PriceNode, PriceHistory (with iterators), Stock, ETF, FinancialAsset
Week 2	CircularQueue, TradeStack, OrderQueue, StockBST, Portfolio, StockManager
Week 3	All 4 TradingStrategy classes, main.cpp menu,

Submission Format

Every student must submit a **single .zip file** named exactly:

LastName_FirstName_StockSim.zip

When unzipped it must produce **exactly this folder structure** — no extra nesting, no renamed folders:

```
LastName_FirstName_StockSim/
|
|— include/                                ← All header files (provided + any you
add)
|   |— FinancialAsset.h
|   |— PriceNode.h
|   |— PriceHistory.h
|   |— CSVParser.h
|   |— Stock.h
|   |— ETF.h
|   |— CircularQueue.h
|   |— TradeStack.h
|   |— OrderQueue.h
|   |— StockBST.h
|   |— Portfolio.h
|   |— TradingStrategy.h
|   |— FixedSIPStrategy.h
|   |— DynamicSIPStrategy.h
|   |— GoldenCrossStrategy.h
|   |— MomentumStrategy.h
|   |— StockManager.h
|
|— src/                                    ← All .cpp implementation files YOU wrote
|   |— FinancialAsset.cpp
|   |— PriceHistory.cpp
|   |— CSVParser.cpp
|   |— Stock.cpp
|   |— ETF.cpp
|   |— CircularQueue.cpp
|   |— TradeStack.cpp
|   |— OrderQueue.cpp
|   |— StockBST.cpp
|   |— Portfolio.cpp
|   |— TradingStrategy.cpp
|   |— FixedSIPStrategy.cpp
|   |— DynamicSIPStrategy.cpp
|   |— GoldenCrossStrategy.cpp
|   |— MomentumStrategy.cpp
```

├─ data/	← CSV files (provided in starter kit – do not replace)
│ ├─ SPX.csv	
│ ├─ AMZN.csv	
│ └─ NVidia_stock_history.csv	
├─ report/	← Written report (max 20 pages)
│ └─ Report.pdf	
├─ output/	← Terminal screenshots of your program
│ ├─ menu_main.png	Screenshot of main menu
│ ├─ strategy_comparison.png	Screenshot of menu item 13 (head-to-head results)
│ ├─ bst_traversal.png	Screenshot of BST
│ ├─ portfolio_summary.png	Screenshot of portfolio holdings display
│ └─ bonus_sweep.png	(Bonus only) Screenshot of parameter sweep output
├─ main.cpp	← Your main program
├─ CMakeLists.txt	← Build file
├─ PROMPTS.md	← Consolidated list of all AI prompts
└─ bonus_strategy.txt	← (Bonus only) Your DynamicSIP parameters
+ results	

Output Screenshots (output/ folder)

The output/ folder is **required**. It proves your code runs correctly. For each screenshot:

- Run your program, navigate to the relevant menu item, and capture the terminal output
- Screenshots must show **real data** loaded from your CSV files — not placeholder/test output
- On Mac: Cmd + Shift + 4 to capture a selection; on Windows: Win + Shift + S
- Acceptable formats: .png, .jpg

Screenshot File	What it must show
menu_main.png	The main menu with your name and student ID displayed
strategy_comparison.png	Menu item 13 — all 4 strategies compared on SPX with final values, CAGR, and max drawdown
bst_traversal.png	Menu item 6 — BST inorder traversal showing at least 5 inserted stocks sorted by

	return
portfolio_summary.png	Menu item 14 — portfolio with at least 2 positions, showing unrealized return
bonus_sweep.png	<i>(Bonus only)</i> Parameter sweep output ranked by BST inorder traversal

Build & Run Instructions

Your submission must compile and run with **one of these two commands** from the project root:

```
# Option A – CMake
mkdir build && cd build && cmake .. && make
./stocksim

# Option B – Direct g++
g++ -std=c++11 -Iinclude src/*.cpp main.cpp -o stocksim
./stocksim
```

If your code requires any additional steps to compile, include a README.txt in the root folder explaining them.

Do not submit: compiled binaries, object files (.o), .DS_Store, or build/ directories.
